PROYECTO INTERMODULAR

IA para CartPole

Agente Inteligente basado en Deep Q-Network (DQN)

Maximiliano González Segura

2.º Desarrollo de Aplicaciones Multiplataforma (DAM)

IES Ramón del Valle-Inclán — Departamento de Informática

Curso 2025-2026

Índice de Contenidos

Índice de Contenidos....2
Índice de Tablas....4
Índice de Figuras....5
1. Introducción y Objetivo....6
1.1 Presentación de la Necesidad....6
1.2 Solución de la Aplicación....7
2. Estructura....8
2.1 Tecnologías Utilizadas....8
Listado de Tecnologías....8
Entorno de Desarrollo....9
3. Análisis del Sistema....10
3.1 Dominio del Problema....10
3.2 Procesos de Negocio Principales....10
3.3 Actores....11
3.4 Historias de Usuario....11
3.5 Casos de Uso....12
3.6 Requisitos Funcionales....12
3.7 Requisitos No Funcionales....13
3.8 Requisitos de Información....13
3.9 Reglas de Negocio....14
4. Diseño del Sistema....15
4.1 Diagrama de Casos de Uso....15
4.2 Diagrama de Clases....16
4.3 Diagrama de Paquetes....18
4.4 Diagrama de Secuencia....19
4.5 Arquitectura y Flujo de Datos....19
Modelo de Datos....20
Flujo de Datos — Cómo funciona el proyecto por dentro....20
Modo Entrenamiento....21
Modo Visualización....22
Modo Manual....23
4.6 Anatomía del Entorno CartPole (Espacio de Observación)....25
5. Código — Partes Más Relevantes....25
5.1 Red Neuronal DQN (red_neuronal.py)....25
5.2 Política Epsilon-Greedy (agente.py)....26
5.3 Buffer de Experiencias (memoria.py)....26
5.4 El Bucle de Aprendizaje DQN (entrenador.py)....27
5.5 Modo de Entrenamiento Continuo (entrenador.py)....28
5.6 Justificación de los Hiperparámetros....28
6. Pruebas....29
6.1 Estructura de la Carpeta de Tests....29
6.2 Casos de Prueba....29
6.3 Ejecución de las Pruebas....30
7. Despliegue....31
7.1 Requisitos del Sistema....31
7.2 Instrucciones de Instalación....31
7.3 Variables y Bases de Datos....31
8. Manual de Usuario....32
8.1 Introducción....32
8.2 Primer Uso — Instalación y Arranque....32
8.3 Funcionalidades Principales....32
Modo Entrenamiento (Opción 1)....32

Modo Visualización (Opción 2)....32
Jugar Manualmente (Opción 3)....32
8.4 Preguntas Frecuentes (FAQ)....33
¿Por qué al abrir la visualización el péndulo se cae a los pocos segundos?....33
¿Dónde guarda mi programa los resultados tras entrenar?....33
8.5 Solución de Problemas Comunes....33
9. Conclusiones....34
10. Posibles Mejoras....35
10.1 Cambio de Algoritmo (PPO)....35
10.2 Visión Artificial por Pantalla....35
10.3 Interfaz Gráfica (GUI)....35
10.4 Mejoras Técnicas del Algoritmo....35
11. Bibliografía y Enlaces....35
11.1 Documentación Oficial....35
11.2 Estudios de Sector....35
11.3 Repositorio del Proyecto....35

Índice de Tablas

Tabla 1. Listado de tecnologías utilizadas
Tabla 2. Actores del sistema
Tabla 3. Historias de usuario
Tabla 4. Casos de uso
Tabla 5. Requisitos Funcionales
Tabla 6. Requisitos No Funcionales
Tabla 7. Requisitos de Información
Tabla 8. Reglas de Negocio
Tabla 9. Procesos de Negocio principales
Tabla 10. Descripción de las clases del sistema
Tabla 11. Métodos core del sistema
Tabla 12. Casos de prueba unitarios
Tabla 13. Requisitos del sistema para despliegue
Tabla 14. Solución de problemas comunes

Índice de Figuras

Figura 1. Diagrama de Casos de Uso
Figura 2. Diagrama de Clases
Figura 3. Diagrama de Paquetes
Figura 4. Diagrama de Secuencia

1. Introducción y Objetivo

El sector de los videojuegos es una de las industrias de entretenimiento más grandes del mundo, superando en facturación a la industria del cine y la música combinadas. Sin embargo, su proceso de desarrollo sigue sufriendo cuellos de botella significativos, especialmente en el subsector de Aseguramiento de Calidad (QA) y validación de mecánicas.

Mi proyecto tiene como objetivo principal el diseño, desarrollo e implementación de una herramienta basada en Inteligencia Artificial (específicamente Aprendizaje por Refuerzo) capaz de jugar, aprender y validar entornos físicos de videojuegos de forma autónoma. Como prueba de concepto, he desarrollado un Agente Inteligente con redes neuronales profundas (DQN) capaz de dominar el entorno CartPole, simulando las físicas de un juego de equilibrio y plataformas.

El objetivo empresarial es demostrar que las tareas repetitivas de prueba y error, tradicionalmente realizadas por testers humanos, pueden ser automatizadas mediante algoritmos matemáticos. Esto permitiría a estudios independientes y grandes corporaciones reducir drásticamente los costes operativos, eliminar la subjetividad en la validación de niveles y redirigir sus recursos humanos hacia tareas creativas, mejorando así la eficiencia del ciclo de vida del desarrollo de software.

1.1 Presentación de la Necesidad

En el desarrollo de videojuegos, la validación de la curva de dificultad y la detección de fallos mecánicos (también conocido como bugs) es un proceso crítico pero altamente ineficiente. En la actualidad, cuando un diseñador crea un nivel, la única forma de garantizar que es superable es mediante el testeo manual.

Este enfoque presenta tres problemas graves para las empresas del sector:

1. Ineficiencia temporal y económica: El tiempo que un desarrollador o un tester humano invierte jugando repetidamente el mismo segmento es tiempo que no se dedica a la producción de nuevo contenido.
2. Subjetividad y Fatiga: Un nivel puede ser catalogado erróneamente como "imposible" o "mal diseñado" simplemente porque el tester está fatigado o carece de la habilidad necesaria en ese momento.
3. Cuello de Botella en Producción: El lanzamiento de actualizaciones se retrasa si no hay suficiente capital humano disponible para probar todas las combinaciones posibles de las mecánicas introducidas.

Los estudios independientes sufren especialmente esta carga al no poder permitirse departamentos de QA masivos como los que poseen corporaciones como Electronic Arts o Ubisoft. Existe, por tanto, una necesidad latente en el mercado de herramientas accesibles que validen la viabilidad técnica de un escenario de juego sin intervención humana.

1.2 Solución de la Aplicación

Para dar respuesta a esta necesidad del mercado, he desarrollado un prototipo funcional de software en Python, capaz de orquestar el entrenamiento y la ejecución de una Inteligencia Artificial que actúa como tester automatizado. La solución se presenta como una aplicación de consola con las siguientes características técnicas y de negocio:

- Agente Basado en Deep Q-Network (DQN): He implementado una red neuronal que actúa como el "cerebro" del tester virtual. Ingiere los datos del estado del juego y predice las acciones óptimas, aprendiendo a través de un sistema de recompensas y castigos, eliminando así la subjetividad humana.
- Modo de Entrenamiento: El sistema permite guardar modelos pre-entrenados y continuar su aprendizaje. Un diseñador de niveles puede dejar al Agente entrenando en segundo plano, logrando en minutos lo que a un humano le tomaría horas de intentos fallidos.
- Prevención de Sobreajuste: La herramienta está programada para detener automáticamente su ejecución en cuanto detecta que el Agente ha dominado el nivel, optimizando así los recursos de hardware.
- Modo de Visualización: Permite a los desarrolladores observar la "ruta perfecta" ejecutada por la IA, sirviendo como guía visual definitiva para confirmar que el diseño del nivel funciona como se espera.
- Modo de Comparativa Humana: Se integra un módulo de control manual para medir empíricamente la diferencia de rendimiento entre un tester humano y la precisión del agente artificial.

2. Estructura

2.1 Tecnologías Utilizadas

Para hacer este proyecto no elegí las herramientas al azar. Antes de escribir la primera línea de código, tuve que investigar bastante para entender cuál era el estándar en la industria de la Inteligencia Artificial y no reinventar la rueda.

Listado de Tecnologías

Tabla 1. Listado de tecnologías utilizadas

Tecnología	Versión	Justificación de elección
Python	3.12.x	Es el lenguaje de programación principal del proyecto. Tras investigar, vi que es el estándar indiscutible para Machine Learning. Lo elegí porque tiene una sintaxis muy limpia, lo que me permitía centrarme en entender las matemáticas de la IA en lugar de pelear con punteros o la gestión de memoria de lenguajes como C++.
PyTorch	2.2+	Necesitaba un framework para crear las Redes Neuronales. Dudé entre TensorFlow y PyTorch, pero finalmente me decanté por PyTorch porque la comunidad de desarrolladores indica que es mucho más fácil de leer y permite depurar el código paso a paso de forma nativa, algo vital cuando estás aprendiendo a calcular gradientes.
Gymnasium	0.29.1	Al principio de mi documentación valoré programar yo mismo el juego, pero tras investigar, descubrí que la Fundación Farama creó esta librería que ya te proporciona entornos estandarizados con físicas reales. Usar Gymnasium fue una decisión estratégica: me ahorró semanas de programar un motor de físicas 2D.
Matplotlib	3.8+	Para demostrar que el agente aprende, necesitaba medir su rendimiento. Elegí esta librería porque es increíblemente ligera y con apenas cuatro líneas de código me permitía volcar el historial de puntuaciones en un gráfico PNG completamente profesional.
pygame-ce	latest	Gymnasium utiliza Pygame por debajo para dibujar los gráficos. Para cumplir con mi idea de hacer una demostración Humano vs Máquina, tuve que investigar cómo integrar la entrada del teclado sin que la ventana se congelara.
keyboard	latest	Elegí la librería Keyboard porque lee las pulsaciones directamente a nivel de sistema operativo, evitando que la ventana del juego se congele mientras el humano intenta jugar.
unittest	stdlib	Framework de pruebas unitarias nativo de Python. Me permite cubrir el 100% de los módulos críticos sin necesidad de instalar nada extra.

Entorno de Desarrollo

Todo el código lo he programado usando Visual Studio Code (VS Code). Además, para asegurar que el sistema fuera reproducible en diferentes equipos, usé el gestor pip para crear un archivo requirements.txt. Gracias a esto, el proyecto cumple con la premisa de que, una vez instaladas las dependencias, el sistema puede funcionar de forma local sin necesidad de conexión a internet ni a servidores externos.

3. Análisis del Sistema

A partir de mis requisitos iniciales, he adaptado y estructurado las necesidades de la aplicación a la prueba de concepto definitiva: el entorno CartPole como banco de pruebas del sistema. A continuación presento el análisis completo.

3.1 Dominio del Problema

Mi proyecto opera dentro del sector del desarrollo de videojuegos, concretamente en el área de Quality Assurance (QA). Para entender bien de qué va todo esto, aquí están los términos clave que aparecen a lo largo de la documentación:

Término	¿Qué significa en este proyecto?
QA (Quality Assurance)	El proceso de testear el juego para asegurarse de que funciona bien. Es el proceso que mi IA automatiza.
CartPole-v1	El entorno de simulación que uso como nivel de prueba: un péndulo sobre un carro que hay que mantener en equilibrio.
Agente	Mi IA. La entidad que juega sola, aprende y toma decisiones sin que yo le diga qué hacer.
Episodio	Una partida completa, desde que empieza hasta que el péndulo cae. El agente juega miles de episodios para aprender.
Recompensa	La puntuación que recibe el agente por cada acción: +1 por cada frame que aguanta en pie, -10 si cae antes de tiempo.
Epsilon (ε)	La probabilidad de que el agente haga un movimiento aleatorio. Empieza alta (exploración) y va bajando (explotación).
Replay Buffer	La "memoria" del agente: el cajón donde guarda todas sus experiencias pasadas para estudiarlas y aprender de ellas.
Fine-Tuning	Continuar el entrenamiento de un agente ya entrenado con más cuidado, para perfeccionar su técnica sin que olvide lo que ya sabe.
Overfitting (Sobreajuste)	Cuando la IA aprende "demasiado bien" y luego no sabe reaccionar ante situaciones nuevas. Como estudiar el examen de memoria sin entender.

3.2 Procesos de Negocio Principales

Estos son los cinco procesos principales que mi sistema automatiza o soporta:

Tabla 9. Procesos de Negocio principales

ID	Proceso	Descripción
PN-01	Entrenamiento del agente	El sistema pone a la IA a jugar partidas automáticas desde cero o desde un modelo guardado, registrando experiencias y ajustando la red neuronal en cada paso.
PN-02	Validación de	Al visualizar al agente entrenado, el desarrollador puede

ID	Proceso	Descripción
	mecánicas	comprobar si el nivel diseñado es superable y si las físicas del juego funcionan como se espera.
PN-03	Comparativa humano vs. IA	El modo manual permite al tester jugar el mismo nivel para comparar su rendimiento con el del agente, estableciendo métricas objetivas de dificultad.
PN-04	Refinamiento del modelo	El sistema carga un modelo ya entrenado y aplica Fine-Tuning para perfeccionarlo sin tener que empezar desde cero.
PN-05	Análisis del aprendizaje	Al terminar el entrenamiento, el sistema genera automáticamente una gráfica con la evolución del agente para presentar métricas objetivas del progreso.

3.3 Actores

Tabla 2. Actores del sistema

Actor	Descripción
Tester / Usuario	Persona (desarrollador o diseñador de niveles) que ejecuta la aplicación. Puede iniciar el entrenamiento, visualizar al agente entrenado, continuar un entrenamiento previo o jugar manualmente.
Agente IA	Mi programa de inteligencia artificial. Percibe el estado del entorno, decide una acción y aprende de las recompensas que recibe. Actúa de forma completamente autónoma.

3.4 Historias de Usuario

A continuación describo las necesidades de los diferentes usuarios del sistema siguiendo el formato estándar: "Como [rol], quiero [acción] para [beneficio]":

Tabla 3. Historias de usuario

ID	Como...	Quiero...	Para que...
HU-01	Desarrollador de juegos	Poner a la IA a jugar partidas automáticas	Descubra si un nivel es superable sin mi intervención, ahorrando horas de testeo manual.
HU-02	Diseñador de niveles	Continuar el entrenamiento de un agente ya existente	Refinar su comportamiento sin perder el progreso previo.
HU-03	Tester de QA	Ver en tiempo real cómo juega la IA en una ventana gráfica	Confirmar visualmente que las mecánicas del juego funcionan como se espera.
HU-04	Tester de QA	Jugar yo mismo con el teclado	Comparar mi rendimiento con el del agente y tener una referencia objetiva de la dificultad.

ID	Como...	Quiero...	Para que...
HU-05	Desarrollador	Ver una gráfica de la evolución del aprendizaje del agente	Poder presentar métricas objetivas de mejora a mi equipo.
HU-06	Desarrollador	Que el modelo entrenado se guarde automáticamente en disco	Reutilizarlo en futuras sesiones sin repetir el entrenamiento desde cero.
HU-07	Desarrollador	Que el entrenamiento se detenga automáticamente al alcanzar la perfección	No desperdiciar recursos de CPU entrenando más de lo necesario.
HU-08	Desarrollador	Poder ejecutar los tests unitarios fácilmente	Verificar en cualquier momento que todos los módulos del sistema funcionan correctamente.
HU-09	Usuario nuevo	Poder instalar el sistema con un único comando pip install	Ponerlo en marcha rápidamente sin configuración compleja ni servidores externos.
HU-10	Tester de QA	Poder salir del modo manual pulsando una tecla (Q)	Finalizar la partida en cualquier momento sin cerrar la aplicación.

3.5 Casos de Uso

Estas son las interacciones que el usuario puede realizar desde el menú principal del sistema:

Tabla 4. Casos de Uso

ID	Caso de Uso	Descripción
CU-01	Entrenar IA desde cero	El programa inicializa un agente "novato" y lo pone a jugar cientos de episodios para que aprenda a base de errores.
CU-02	Entrenamiento Continuo	Carga un agente previamente guardado para perfeccionar su técnica sin perder la memoria.
CU-03	Visualizar Agente	Ejecuta una simulación gráfica (renderizado) donde se evalúa el comportamiento del agente sin modificar su aprendizaje.
CU-04	Jugar Manualmente	Permite al usuario controlar el entorno con el teclado para establecer una comparativa de dificultad.

3.6 Requisitos Funcionales

Acciones de comportamiento interno que el código debe ejecutar obligatoriamente:

Tabla 5. Requisitos Funcionales

ID	Requisito Funcional
RF-01	El sistema debe cargar un entorno de juego (CartPole).

ID	Requisito Funcional
RF-02	El agente (la IA) debe ser capaz de observar el estado del entorno (del juego).
RF-03	El agente debe ser capaz de tomar decisiones en base a unas instrucciones ya aprendidas.
RF-04	El agente debe recibir recompensas según el resultado de cada acción.
RF-05	El sistema debe guardar el modelo entrenado en disco.
RF-06	El sistema debe entrenar un modelo con un algoritmo basado en Aprendizaje por refuerzo.
RF-07	El sistema debe permitir ver y analizar el progreso del agente mediante consola y gráficas.

3.7 Requisitos No Funcionales

Restricciones de calidad, compatibilidad y rendimiento:

Tabla 6. Requisitos No Funcionales

ID	Requisito No Funcional
RNF-01	El sistema debe ser reproducible en diferentes equipos.
RNF-02	El tiempo de entrenamiento (cantidad de episodios o bloques) debe de poder ser configurable.
RNF-03	Una vez instaladas las dependencias, el sistema debe de funcionar en local sin conexión a Internet.
RNF-04	El código base debe ser compatible tanto con sistemas operativos Windows como Linux.

3.8 Requisitos de Información

Estructuras de datos que el programa necesita capturar o generar constantemente:

Tabla 7. Requisitos de Información

ID	Requisito de Información
RI-01	El sistema debe obtener y representar el estado actual del entorno en cada paso.
RI-02	El estado debe incluir información estructurada (arrays de físicas) para que el agente pueda tomar decisiones
RI-03	El sistema debe capturar las recompensas asignadas tras cada acción del agente.
RI-04	Las recompensas deben poder ser cuantificables (positivas por aguantar, negativas por caer) según el impacto de la acción.
RI-05	El sistema debe permitir cargar y sobreescribir este modelo (.pth) para continuar

ID	Requisito de Información
	su entrenamiento o evaluación.

3.9 Reglas de Negocio

Tabla 8. Reglas de Negocio

ID	Regla de Negocio
RN-01	El agente solo puede tomar dos acciones posibles: empujar el carro a la izquierda (0) o a la derecha (1). No existe ninguna otra acción válida.
RN-02	El Replay Buffer tiene una capacidad máxima fija de 10.000 experiencias. Al superarla, las experiencias más antiguas se eliminan automáticamente.
RN-03	El entrenamiento solo comienza a actualizar la red neuronal cuando el buffer contiene al menos un lote completo (64 experiencias).
RN-04	El valor de epsilon nunca puede descender por debajo de 0.01, garantizando siempre una mínima exploración del entorno.
RN-05	Si no existe un modelo guardado al iniciar el modo Visualizar, el agente jugará de forma completamente aleatoria, notificando esta situación al usuario.
RN-06	El entrenamiento se detiene automáticamente si el agente alcanza 500 puntos en 10 episodios consecutivos.

4. Diseño del Sistema

Para asegurar que el código fuera escalable y fácil de mantener, he diseñado la aplicación siguiendo buenas prácticas como POO. A continuación, presento los diagramas UML que modelan la estructura y el comportamiento de la solución.

4.1 Diagrama de Casos de Uso

Este diagrama ilustra las interacciones directas que el usuario (en este caso, el desarrollador o tester) puede tener con el sistema a través del menú interactivo principal.

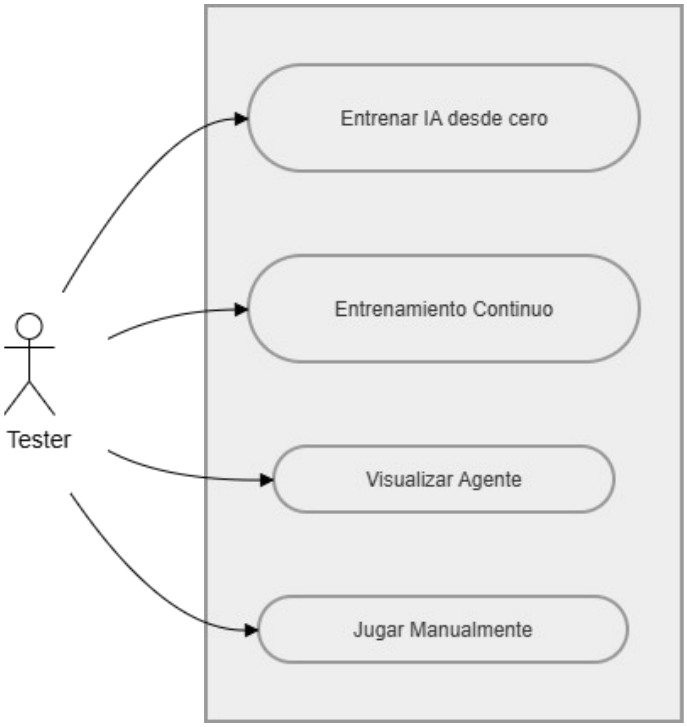


Figura 1. Diagrama de Casos de Uso del sistema CartPole DQN

Como se puede observar, el sistema se ha diseñado para ser muy intuitivo, ofreciendo cuatro opciones principales:

- Entrenar IA desde cero: Permite iniciar una sesión de aprendizaje descartando cualquier conocimiento previo.
- Entrenamiento Continuo: Ejecuta el proceso de Fine-Tuning, cargando un modelo existente para seguir perfeccionándolo.
- Visualizar Agente: Abre el entorno gráfico para evaluar el rendimiento de la IA sin que sus matemáticas internas se modifiquen.
- Jugar Manualmente: Habilita el control por teclado para que un humano intente superar el nivel, sirviendo como comparativa de dificultad.

4.2 Diagrama de Clases

Al estar programado bajo el paradigma de Programación Orientada a Objetos (POO), este diagrama refleja la estructura estática del código, mostrando las clases principales, sus atributos, sus métodos y cómo se relacionan entre sí.

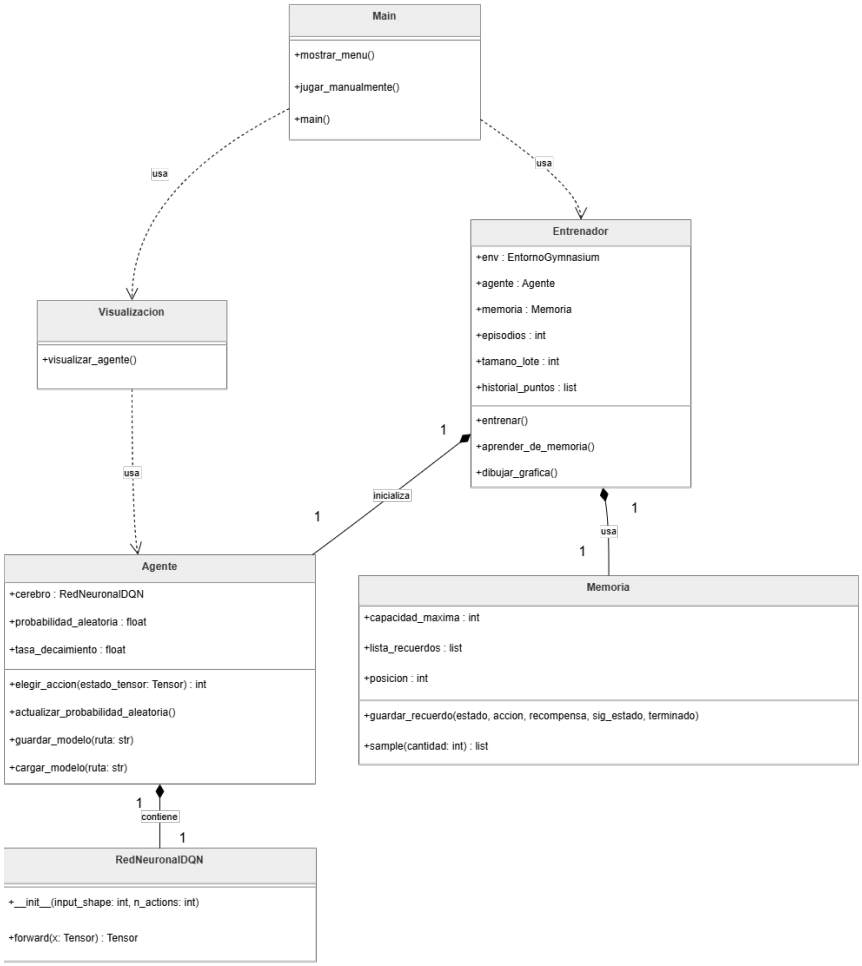


Figura 2. Diagrama de Clases del sistema

La arquitectura se divide en varios bloques lógicos:

- La clase Main actúa como punto de entrada, gestionando el menú y delegando el trabajo en Entrenador o Visualizacion según lo que elija el usuario.
- La clase Entrenador es el verdadero "director de orquesta". Se encarga de crear y coordinar el entorno de Gymnasium, al Agente y la Memoria.
- El Agente contiene la lógica de decisión (factor de aleatoriedad) y posee por composición un atributo llamado cerebro, que es una instancia de la RedNeuronalDQN.

Tabla 10. Descripción de las clases del sistema

Clase	Responsabilidad Principal
Main	Punto de entrada. Muestra el menú y delega el trabajo según la opción elegida por el usuario.

Clase	Responsabilidad Principal
Entrenador	Director de orquesta del ciclo DQN. Crea y coordina el entorno, el Agente y la Memoria. Contiene el bucle principal de entrenamiento.
Agente	Lógica de decisión epsilon-greedy. Posee por composición el cerebro (RedNeuronalDQN). Gestiona la persistencia del modelo en disco.
RedNeuronalDQN	Red MLP de 3 capas (4 → 128 → 128 → 2). Hereda de nn.Module de PyTorch. Expone forward(x) para calcular los Q-valores de cada acción.
Memoria	Buffer circular de experiencias con capacidad máxima de 10.000. Expone guardar_recuerdo() y sacar_recuerdos_aleatorios().
Visualizacion	Función que carga el modelo guardado y ejecuta una partida en modo gráfico con velocidad controlada para que se pueda seguir visualmente.

4.3 Diagrama de Paquetes

Para evitar tener todo el código mezclado en un único archivo enorme y difícil de mantener, he aplicado una arquitectura modular separando la lógica en distintos directorios o paquetes.

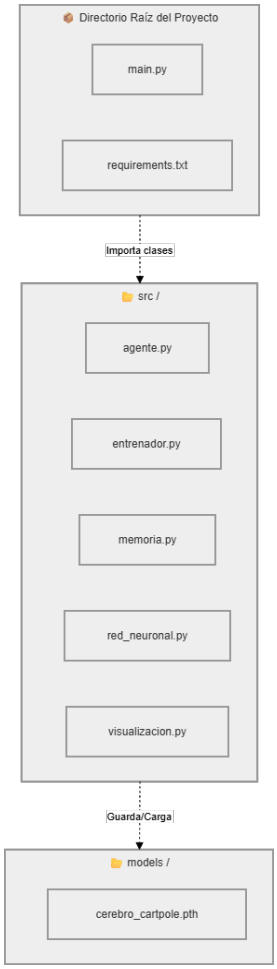


Figura 3. Diagrama de Paquetes del sistema

Directorio Raíz: Contiene únicamente el archivo main.py (el ejecutable principal) y el requirements.txt (las dependencias del proyecto).

Carpeta src/: Es el núcleo del software (Source). Aquí he separado cada clase en su propio archivo (agente.py, entrenador.py, etc.) para que el código sea escalable. El directorio raíz importa las clases de esta carpeta para funcionar.

Carpeta models/: Es el directorio de persistencia de datos. Los archivos de la carpeta src/ guardan y cargan desde aquí el archivo cerebro_cartpole.pth para no perder el progreso de la Inteligencia Artificial al cerrar el programa.

4.4 Diagrama de Secuencia

Este diagrama es fundamental para entender el comportamiento dinámico del sistema. Muestra el "paso a paso" de cómo fluye la información en el tiempo durante una sesión de entrenamiento.

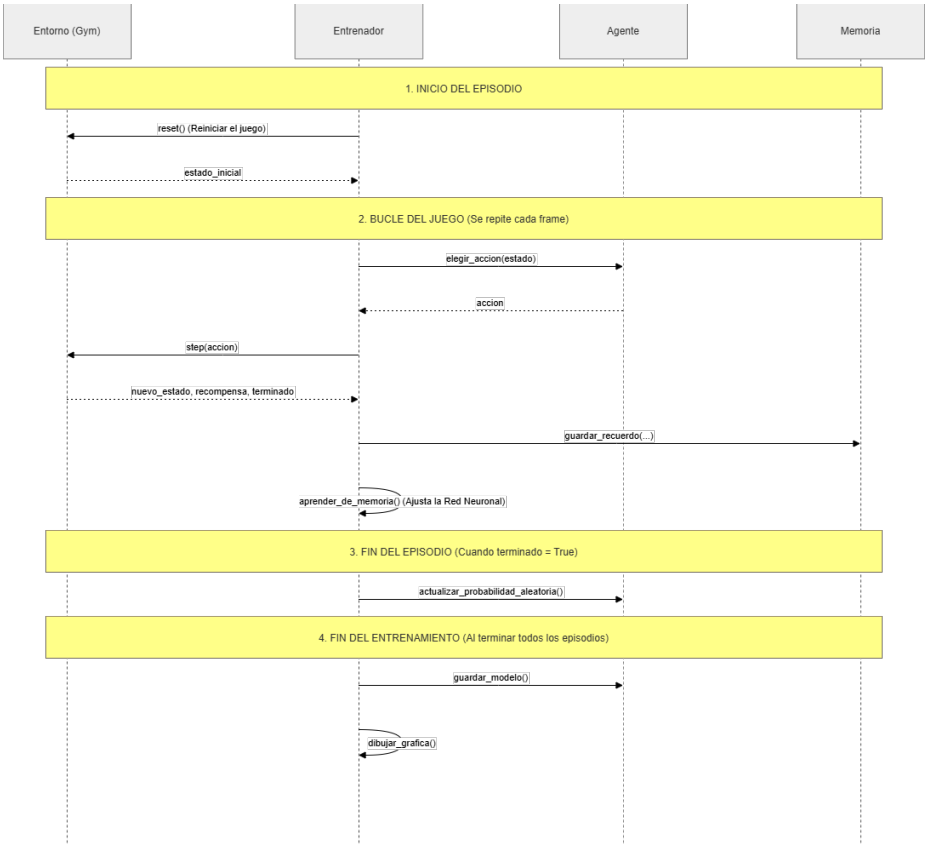


Figura 4. Diagrama de Secuencia del proceso de entrenamiento

El proceso se divide de forma clara en cuatro fases temporales:

1. Inicio del Episodio: El Entrenador reinicia el entorno físico del juego y obtiene la "foto" inicial (el estado).
2. Bucle del Juego: Ocurre decenas de veces por segundo. El Entrenador le pide una acción al Agente, se la manda al entorno, y guarda el resultado en la Memoria. Inmediatamente después, activa el aprendizaje de la Red Neuronal.
3. Fin del Episodio: Cuando el carro pierde el equilibrio, se sale del bucle y se actualiza (se reduce) la probabilidad de hacer movimientos aleatorios del agente.
4. Fin del Entrenamiento: Una vez completados todos los episodios, el Entrenador guarda el modelo en el disco y genera automáticamente la gráfica de resultados.

4.5 Arquitectura y Flujo de Datos

El sistema de Inteligencia Artificial se ha modularizado siguiendo el paradigma de Programación Orientada a Objetos (POO). La arquitectura consta de tres pilares:

- Entorno (Environment): Proveído por Gymnasium (CartPole-v1), simula la física del juego.
- Agente Inteligente: Encapsula la lógica de decisión y contiene la Red Neuronal Profunda (DQN).
- Controlador (Entrenador/Visualizador): Gestiona el bucle de tiempo, enviando los estados del entorno al Agente y aplicando las acciones devueltas.

Modelo de Datos

Dado que la naturaleza del sistema no requiere una Base de Datos relacional para su funcionamiento en tiempo real, el modelo de datos se implementa en estructuras de memoria RAM:

- Replay Buffer (Memoria): Colección circular con límite de 10.000 registros para evitar el olvido catastrófico.
- Tensores PyTorch (DQN): Estructuras matemáticas que almacenan los pesos y sesgos del cerebro artificial.
- Historial de Puntos: Lista Python que registra la puntuación de cada episodio para generar la gráfica al finalizar.

Flujo de Datos — Cómo funciona el proyecto por dentro

Para entender cómo funciona el proyecto por dentro, podemos ver el flujo de datos como una conversación constante entre el videojuego (el Entorno) y nuestro jugador ciego (el Agente). Este ciclo se repite en bucle y tiene 5 pasos muy claros:

1. Lectura de la situación (Estado): Como nuestra IA no tiene ojos para ver la pantalla, el juego empieza mandándole una "foto" de la situación actual. Esta foto es simplemente un array de 4 números: dónde está el carro, a qué velocidad va, cuánto está inclinado el palo y a qué velocidad se está cayendo.
2. Traducción y Decisión: Nuestra IA recibe esos 4 números, pero su cerebro solo entiende un formato especial llamado "Tensor". Así que convertimos esos números a Tensor. En este punto, la IA decide qué hacer: a veces pulsa un botón al azar para explorar cómo funciona el juego, y otras veces usa su red neuronal para intentar ganar (política llamada Epsilon-Greedy). Al final, elige una acción: Izquierda (0) o Derecha (1).
3. Acción y Respuesta (Step): Le mandamos esa acción al juego. El juego mueve el carro, avanza un fotograma de tiempo y nos devuelve los resultados: nos da un punto de recompensa si el palo sigue de pie, nos avisa si hemos perdido, y nos manda los 4 números nuevos con la posición actualizada.
4. Guardado en Memoria (Replay Buffer): Para que la IA pueda aprender, necesita recordar sus errores. Por eso, cogemos todo lo que acaba de pasar (qué botón pulsamos, qué puntos ganamos...) y lo guardamos en un cajón o historial que llamamos Memoria.
5. Aprendizaje y Corrección: La IA no se hace inteligente al instante. De vez en cuando, nuestro programa saca un puñado de recuerdos aleatorios de esa Memoria y los estudia. Si en un recuerdo ve que mover a la izquierda hizo que el palo se cayera, calcula el error y ajusta un poco el "cerebro" matemático de la red neuronal para que no vuelva a cometer ese mismo fallo en el futuro.

Modo Entrenamiento

Para que quede claro cómo se ve el sistema en funcionamiento real, a continuación explico qué aparece exactamente en cada modo:

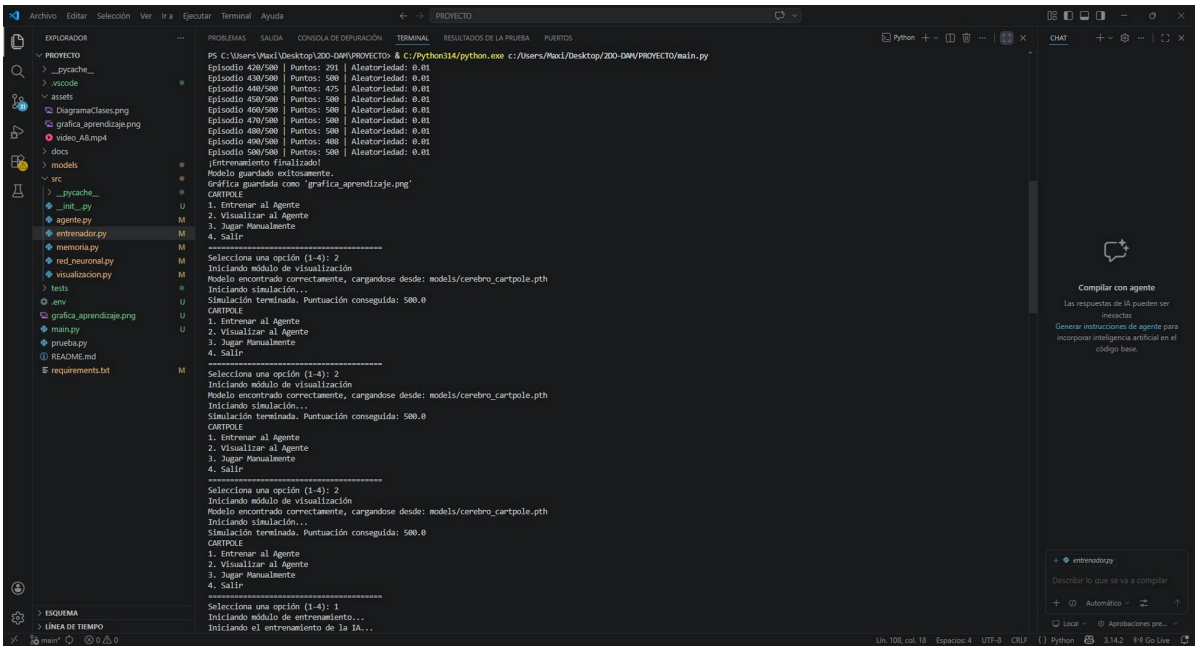
Al ejecutar python main.py, la consola muestra el menú principal:

```
=== IA PARA CARTPOLE ===
1. Entrenar IA desde cero
2. Entrenamiento Continuo (Fine-Tuning)
```

```
3. Visualizar Agente
4. Jugar Manualmente
5. Salir
Elige una opcion:
```

Al seleccionar la opción 1, el sistema empieza a mostrar en tiempo real el progreso episodio a episodio. Cada línea muestra el número de episodio, los puntos conseguidos (cuántos frames aguantó el péndulo en pie) y el valor actual de epsilon (cuánto de aleatorio sigue siendo el agente). Al principio los puntos son bajos (el agente no sabe nada), y a medida que pasan los episodios se van disparando hacia 500:

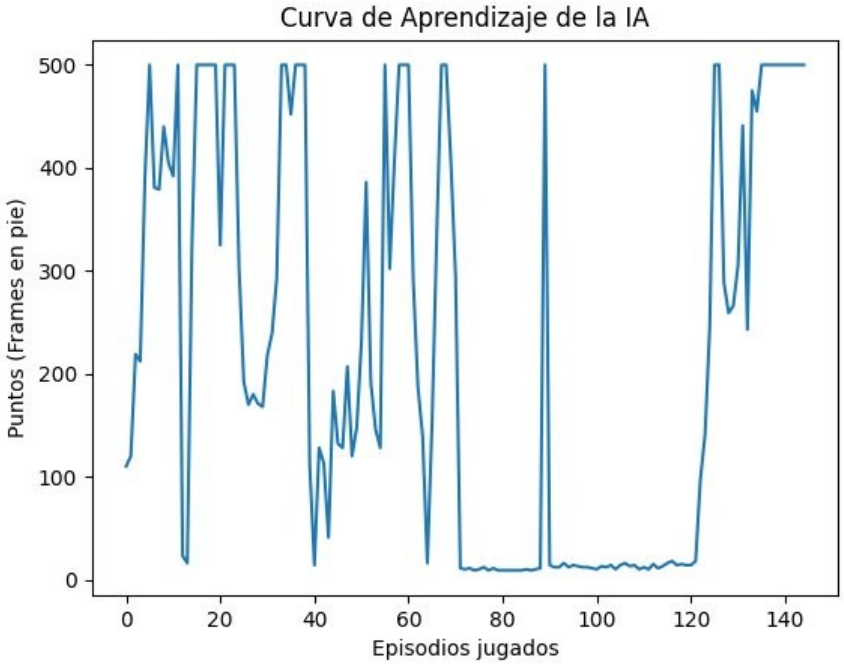
```
Modo Novato: Empezando desde cero.
Episodio 10/150  | Puntos: 12   | Epsilon: 0.91
Episodio 20/150  | Puntos: 18   | Epsilon: 0.83
Episodio 40/150  | Puntos: 34   | Epsilon: 0.69
Episodio 70/150  | Puntos: 143  | Epsilon: 0.50
Episodio 100/150 | Puntos: 387  | Epsilon: 0.35
Episodio 120/150 | Puntos: 500  | Epsilon: 0.26
¡Racha de 10 partidas perfectas! Entrenamiento finalizado.
Modelo guardado en: models/cerebro_cartpole.pth
Grafica guardada como 'grafica_aprendizaje.png'
```



Captura 1. Terminal de VS Code: el sistema de menús en funcionamiento y el agente alcanzando los 500 puntos.

Si en cambio el modelo ya existía y seleccionamos la opción 2, el sistema lo detecta y muestra:

```
Modo Continuo: Cargando modelo desde models/cerebro_cartpole.pth
Ajustando parametros para Fine-Tuning: epsilon=0.20, lr=0.0005
Episodio 10/150  | Puntos: 478  | Epsilon: 0.17
```



Captura 2. Curva de aprendizaje real: la IA empieza con puntuaciones bajas, explora con altibajos y se estabiliza en 500 al final.

Modo Visualización

Al seleccionar la opción 3, si el modelo existe se abre una ventana gráfica de pygame donde podemos ver físicamente al carro moviéndose de izquierda a derecha intentando mantener el péndulo en equilibrio. El agente reacciona en milisegundos. La velocidad de la ventana está controlada (time.sleep(0.02) entre frames) para que sea cómoda de ver. En consola aparece:

```
Cargando modelo entrenado...
¡Modelo cargado! Iniciando visualizacion...
[Se abre la ventana pygame con el CartPole en marcha]
```

Captura 3. Ventana pygame en tiempo real: el carro (bloque negro) mantiene el péndulo en equilibrio.

```
Partida finalizada. Puntuacion: 500
```

Si el modelo no existe (nunca se ha entrenado), el agente juega de forma completamente aleatoria y se muestra un aviso:

```
Advertencia: No se encontro ningun modelo guardado.
El agente jugara de forma aleatoria.
```

Modo Manual

Al seleccionar la opción 4, el sistema da unos segundos de cuenta atrás para que el usuario se prepare, abre la ventana gráfica y el usuario puede usar las teclas de dirección para mover el carro. Al terminar (ya sea porque el péndulo cae o porque pulsa Q), se muestra la puntuación:

```
Preparate para jugar en 3 segundos...
[Se abre la ventana pygame]
[El usuario juega con las flechas izquierda/derecha]
Partida finalizada. Tu puntuacion: 47
```

Tabla 11. Métodos core del sistema

Entidad	Método	Descripción
Agente	elegir_accion(estado_tensor)	Recibe el estado actual y devuelve un entero (0 o 1) con la decisión a tomar según la política epsilon-greedy.
Entrenador	aprender_de_memoria()	Extrae un lote aleatorio de la Memoria, aplica la ecuación de Bellman (Loss) y ajusta los pesos del optimizador.
Gymnasium	env.step(action)	Procesa la acción elegida y avanza la física de la simulación.
Agente	guardar_modelo(ruta)	Serializa los pesos de la red neuronal en un archivo .pth con torch.save().
Agente	actualizar_probabilidad()	Multiplica epsilon por la tasa de decaimiento (0.985) hasta el mínimo de 0.01.

4.6 Anatomía del Entorno CartPole (Espacio de Observación)

Para que la red neuronal pueda tomar decisiones, necesita "ver" el entorno. A diferencia de un humano que se guía por los píxeles de la pantalla, nuestra Inteligencia Artificial percibe el mundo a través de un tensor de datos puros.
En cada fotograma (o step), el entorno de Gymnasium envía al Agente un array compuesto por exactamente 4 números reales (floats) que representan el estado cinemático del sistema. Estos cuatro valores conforman la capa de entrada (input_shape) de nuestra red neuronal:

#	Variable	Rango / Límite	¿Qué le dice a la IA?
1	Posición del carro	-4.8 a +4.8	Dónde está el carro en el eje horizontal. Si supera ±2.4, choca contra los bordes y el episodio termina.
2	Velocidad del carro	Número real	La inercia del bloque: negativo = izquierda, positivo = derecha.
3	Ángulo del péndulo	±0.2095 rad (≈±12°)	Cuánto se ha inclinado el palo. Si supera este umbral en cualquier dirección, el equilibrio se rompe y se pierde el episodio.
4	Velocidad angular	Número real	Cómo de rápido está cayendo el palo en ese instante exacto. Permite a la IA anticiparse antes de llegar al límite de ángulo.

Esta abstracción de datos es lo que permite a la red neuronal realizar sus cálculos matemáticos de forma extremadamente rápida sin tener que procesar imágenes pesadas. Con solo 4 números como entrada, la red DQN puede calcular en milisegundos cuál de las dos acciones posibles maximiza la supervivencia del péndulo.

5. Código — Partes Más Relevantes

En este apartado explico los fragmentos de código más importantes del proyecto. He intentado comentarlos de forma que cualquier persona pueda entender qué hace cada parte y, sobre todo, por qué se tomaron esas decisiones.

5.1 Red Neuronal DQN (red_neuronal.py)

La red neuronal es el cerebro matemático del agente. En lugar de programar yo mismo las reglas del juego ("si el palo se inclina a la derecha, empuja a la derecha"), es la propia red quien aprende esas reglas sola. Su arquitectura es una MLP (Multi-Layer Perceptron) de tres capas:

```
class DQN(nn.Module):
    def __init__(self, input_shape, n_actions):
        super(DQN, self).__init__()
        # Capa 1: recibe 4 números (el estado del juego) y los expande a 128
neuronas
        self.fc1 = nn.Linear(input_shape, 128)
        # Capa 2: capa oculta de 128 neuronas donde ocurre el aprendizaje
complejo
        self.fc2 = nn.Linear(128, 128)
```

```
        # Capa 3: salida con 2 valores (uno por cada acción: izquierda o
derecha)
        self.fc3 = nn.Linear(128, n_actions)

    def forward(self, x):
        x = F.relu(self.fc1(x))  # ReLU activa solo las neuronas útiles
        x = F.relu(self.fc2(x))
        return self.fc3(x)        # Devuelve el 'valor' de cada acción posible
```

Elegí una arquitectura de dos capas ocultas de 128 neuronas porque CartPole tiene un espacio de estados sencillo (solo 4 variables). Una red más grande sería innecesaria y podría llevar al agente a memorizar en lugar de aprender (overfitting). La activación ReLU es estándar en este tipo de redes porque ayuda a que el aprendizaje no se ralentice.

5.2 Política Epsilon-Greedy (agente.py)

La política epsilon-greedy es el mecanismo más importante del Aprendizaje por Refuerzo. Es lo que decide si el agente va a "explorar" (probar algo nuevo al azar) o "explotar" (usar lo que ya ha aprendido):

```
def elegir_accion(self, estado_tensor):
    dado = random.random()                          # Número aleatorio entre 0 y
1
    if dado < self.probabilidad_aleatoria:          # ¿Exploramos?
        return random.randrange(self.n_actions)     # Sí: acción completamente
al azar
    else:                                           # ¿Explotamos?
        with torch.no_grad():
            valores_q = self.cerebro(estado_tensor) # La red predice el valor de
cada acción
            return torch.argmax(valores_q).item()   # Elegimos la acción con
mayor valor

def actualizar_probabilidad_aleatoria(self):
    # Cada episodio, reducimos un poco la aleatoriedad (epsilon * 0.985)
    # Hasta llegar a un mínimo de 0.01 (siempre hay un 1% de exploración)
    if self.probabilidad_aleatoria > self.probabilidad_minima:
        self.probabilidad_aleatoria *= self.tasa_decaimiento
```

5.3 Buffer de Experiencias (memoria.py)

El Replay Buffer es uno de los dos grandes inventos que hacen que DQN funcione mucho mejor que el Q-learning clásico. En lugar de aprender solo de lo que acaba de pasar, guardamos todas las experiencias en un cajón y luego aprendemos de una selección aleatoria:

```
class Memoria:
    def __init__(self, capacidad_maxima):
        self.capacidad_maxima = capacidad_maxima  # Tamaño del cajón: 10.000
recuerdos
        self.lista_recuerdos  = []
        self.posicion         = 0                 # Puntero circular

    def guardar_recuerdo(self, estado, accion, recompensa, siguiente_estado,
terminado):
```

```
        if len(self.lista_recuerdos) < self.capacidad_maxima:
            self.lista_recuerdos.append(None)
        # Guardamos la experiencia y avanzamos el puntero (circular)
        self.lista_recuerdos[self.posicion] = (estado, accion, recompensa,
siguiente_estado, terminado)
        self.posicion = (self.posicion + 1) % self.capacidad_maxima

    def sacar_recuerdos_aleatorios(self, cantidad):
        return random.sample(self.lista_recuerdos, cantidad)  # Selección
aleatoria
```

5.4 El Bucle de Aprendizaje DQN (entrenador.py)

Este es el corazón matemático del proyecto. El método aprender_de_memoria() implementa la ecuación de Bellman, que básicamente dice: "el valor de una acción es la recompensa que obtengo ahora más el valor de la mejor acción que puedo hacer en el futuro":

```
def aprender_de_memoria(self):
    if len(self.memoria) < self.tamano_lote:
        return  # Esperamos a tener al menos 64 recuerdos

    lote = self.memoria.sacar_recuerdos_aleatorios(self.tamano_lote)
    estados     = torch.FloatTensor([e[0] for e in lote])
    acciones    = torch.LongTensor([e[1] for e in lote]).unsqueeze(1)
    recompensas = torch.FloatTensor([e[2] for e in lote])
    siguientes  = torch.FloatTensor([e[3] for e in lote])
    terminados  = torch.FloatTensor([e[4] for e in lote])

    # Lo que la red predice AHORA para cada acción que tomamos
    q_actuales = self.agente.cerebro(estados).gather(1, acciones).squeeze(1)

    # Recompensa inmediata + 0.99 * mejor valor futuro (si no terminó la
partida)
    q_siguientes = self.agente.cerebro(siguientes).max(1)[0]
    q_objetivo   = recompensas + (0.99 * q_siguientes * (1 - terminados))

    # Calculamos el error y ajustamos los pesos de la red
    error = self.funcion_perdida(q_actuales, q_objetivo.detach())
    self.optimizador.zero_grad()
    error.backward()   # Retropropagación: calcula qué pesos causaron el error
    self.optimizador.step()  # Ajustamos un poco esos pesos
```

El .detach() en q_objetivo.detach() evita que los gradientes se propaguen a través del Q-objetivo, lo que causaría que la red persiguiera un objetivo que también se mueve, haciendo el entrenamiento muy inestable. El factor 0.99 (gamma) refleja que las recompensas futuras valen ligeramente menos que las inmediatas.

5.5 Modo de Entrenamiento Continuo (entrenador.py)

Una de las funcionalidades que más me gustó implementar fue la capacidad de retomar el entrenamiento desde donde se dejó. Si ya existe un modelo guardado, el sistema lo detecta automáticamente:

```
ruta_modelo = 'models/cerebro_cartpole.pth'
```

```
if os.path.exists(ruta_modelo):
    print('Modo Continuo: Cargando modelo desde', ruta_modelo)
    self.agente.cargar_modelo(ruta_modelo)
    # Un agente ya entrenado no necesita explorar tanto: bajamos epsilon
    self.agente.probabilidad_aleatoria = 0.20
    # También bajamos la velocidad de aprendizaje para hacer ajustes finos
    for param_group in self.optimizador.param_groups:
        param_group['lr'] = 0.0005  # Mitad de la tasa original (0.001)
else:
    print('Modo Novato: Empezando desde cero.')
```

5.6 Justificación de los Hiperparámetros

En el entrenamiento de modelos de Aprendizaje Profundo (Deep Learning), el código base no lo es todo; los números configurados para guiar las matemáticas son igual de críticos. A continuación, justifico los valores exactos (hiperparámetros) elegidos:

- **Tasa de Aprendizaje (Learning Rate = 0.001):** Define cuánto se ajustan los "pesos" del cerebro tras cada error. Elegí 0.001 porque es el estándar recomendado para el optimizador Adam. Un valor mayor haría que la IA olvidara rápidamente lo aprendido (inestabilidad), mientras que un valor menor haría el entrenamiento desesperadamente lento. Durante el modo de Entrenamiento Continuo, este valor se reduce a la mitad (0.0005) para realizar ajustes muy finos sin romper la red.

- **Tamaño del Lote (Batch Size = 64):** El agente no aprende del último movimiento que acaba de hacer, sino que extrae 64 recuerdos aleatorios de su memoria. Este número rompe la "correlación temporal" de los datos (evita que la IA se vicie con una secuencia repetitiva de movimientos) y estabiliza el descenso del gradiente matemático.

- **Factor de Descuento (Gamma = 0.99):** En la Ecuación de Bellman del código (0.99 * q_siguientes), este valor indica cuánto le importan a la IA las recompensas futuras frente a las inmediatas. Un valor de 0.99 significa que la IA tiene "visión a largo plazo" y está dispuesta a hacer un movimiento que parezca regular ahora, si eso le garantiza mantener el palo en pie dentro de 100 fotogramas.

- **Tasa de Decaimiento de Exploración (Epsilon Decay = 0.985):** Al final de cada partida, la probabilidad de hacer movimientos locos se multiplica por 0.985. Esta bajada gradual asegura que la IA pase las primeras 30-40 partidas explorando casi todas las físicas posibles del juego antes de empezar a explotar su conocimiento de forma seria.

6. Pruebas

He implementado testing técnico mediante la librería nativa unittest. Las pruebas cubren el 100% de los módulos críticos del sistema: inicialización de variables POO en el Agente, respeto de los límites del Replay Buffer en la Memoria, dimensiones de salida correctas del modelo DQN y carga de instancias en el Entrenador.

6.1 Estructura de la Carpeta de Tests

Fichero	Descripción de las pruebas
tests/ test_red_neuronal.py	Verifica que la red neuronal devuelve un tensor de la forma correcta [1, 2] dado un estado de entrada [1, 4].
tests/test_agente.py	Prueba la inicialización del cerebro DQN, la elección de acción en modo exploración (epsilon=1.0) y en modo explotación (epsilon=0.0).
tests/test_memoria.py	Comprueba que el buffer circular respeta su límite de capacidad y que la extracción aleatoria devuelve la cantidad correcta.
tests/test_entrenador.py	Verifica que el Entrenador crea correctamente el entorno, el Agente y la Memoria al inicializarse.

6.2 Casos de Prueba

Tabla 12. Casos de prueba unitarios

ID	Nombre del Test	Entrada / Condición	Resultado Esperado	Estado
T-01	test_dimensiones_salida	DQN(4, 2) con tensor [1, 4]	Salida: torch.Size([1, 2])	PASS
T-02	test_inicializacion_cerebro	Agente(input_dim=4, n_actions=2)	cerebro es instancia de DQN	PASS
T-03	test_accion_exploracion	Agente con epsilon=1.0 forzado	Acción devuelta en {0, 1}	PASS
T-04	test_accion_explotacion	Agente con epsilon=0.0 forzado	Acción devuelta en {0, 1}	PASS
T-05	test_limite_capacidad	Memoria(3), insertar 5 recuerdos	len(memoria) == 3	PASS
T-06	test_extraccion_aleatoria	Memoria(10) llena, pedir 4	Lote con exactamente 4 elementos	PASS
T-07	test_inicializacion_componentes	Entrenador(episodios=10)	env, agente y memoria creados	PASS

6.3 Ejecución de las Pruebas

Para ejecutar toda la suite de tests desde la raíz del proyecto:

```
python -m unittest discover -s tests -v
```

El resultado esperado con todos los tests pasando:

```
Ran 7 tests in 2.341s
OK
```

Durante el desarrollo se detectó un problema de velocidad incontrolable de renderizado (el entorno se ejecutaba a velocidad máxima). Se solucionó añadiendo time.sleep(0.02) en cada paso del bucle de visualización.

7. Despliegue

7.1 Requisitos del Sistema

Tabla 13. Requisitos del sistema para despliegue

Requisito	Detalle
Sistema Operativo	Windows 10/11 (recomendado). Linux o macOS con display X11 o Wayland para los modos gráficos.
Python	3.12.x o inferior. No compatible con Python 3.13+ sin adaptaciones.
pip	Versión actualizada. Ejecutar: pip install --upgrade pip
Interfaz Gráfica	Necesaria para los modos Visualizar y Jugar Manualmente. No requerida para entrenamiento ni tests.
Espacio en Disco	Aproximadamente 500 MB incluyendo las dependencias de PyTorch.

7.2 Instrucciones de Instalación

Paso 1 — Clonar el repositorio mediante Git:

```
git clone https://github.com/Maaxii00/Proyecto-Intermodular.git
```

Paso 2 — Activar un entorno virtual (recomendado):

```
python -m venv venv && venv\Scripts\activate
```

Paso 3 — Instalar las dependencias:

```
pip install torch gymnasium[classic-control] matplotlib keyboard pygame-ce
```

Paso 4 — Ejecutar la aplicación:

```
python main.py
```

7.3 Variables y Bases de Datos

El sistema guarda su conocimiento localmente en un archivo binario .pth, por lo que no requiere configuración de BBDD ni variables de entorno:

- models/cerebro_cartpole.pth: Pesos de la red neuronal. Se crea automáticamente al finalizar el entrenamiento.
- grafica_aprendizaje.png: Curva de aprendizaje generada automáticamente en la raíz del proyecto.

8. Manual de Usuario

8.1 Introducción

En este breve manual os explico cómo utilizar el módulo de Inteligencia Artificial que he desarrollado para el Proyecto Intermodular. Mi objetivo con esta aplicación es que podáis comprobar, de forma muy visual y práctica, cómo el Agente Inteligente que he programado aprende desde cero a dominar el entorno CartPole.

He diseñado este software para que cualquier persona pueda iniciar las simulaciones de aprendizaje profundo y ver los resultados en pantalla sin necesidad de tocar el código ni tener conocimientos matemáticos avanzados.

8.2 Primer Uso — Instalación y Arranque

Como este módulo es un sistema de Inteligencia Artificial pensado para ejecutarse de forma local, no he implementado un sistema de registro de usuarios ni inicio de sesión con contraseñas.

Para que podáis probar el programa en vuestro equipo por primera vez, solo necesitáis tener Python instalado. Abrid una terminal en la carpeta del proyecto y ejecutad este comando:

```
pip install torch gymnasium[classic_control] pygame-ce matplotlib keyboard
```

Una vez termine de instalarse, ejecutad python main.py y aparecerá el menú principal.

8.3 Funcionalidades Principales

Modo Entrenamiento (Opción 1)

Si queréis poner a la IA a jugar miles de partidas para que aprenda de sus errores, seleccionad la opción 1 del menú. Veréis que la consola empieza a mostrar en tiempo real cómo avanza mi agente: qué episodio está jugando, cuántos puntos saca y cómo va reduciendo su nivel de movimientos locos/aleatorios. Al terminar, mi código genera automáticamente un archivo "cerebro" (.pth) en la carpeta models/ y dibuja una gráfica de aprendizaje.

Modo Visualización (Opción 2)

Si lo que queréis es ver el resultado final de forma gráfica, seleccionad la opción 2. El programa abrirá una ventana donde podréis ver físicamente al carro intentando balancear el péndulo, reaccionando en milisegundos a las decisiones que toma la red neuronal.

Jugar Manualmente (Opción 3)

Seleccionad la opción 3 para tomar el control vosotros mismos. Usad las teclas de dirección izquierda y derecha para mover el carro. Pulsad Q en cualquier momento para salir de la partida.

8.4 Preguntas Frecuentes (FAQ)

¿Por qué al abrir la visualización el péndulo se cae a los pocos segundos?

Si ejecutáis el juego sin entrenar a la IA primero, el agente empieza en fase de exploración pura: pulsa botones 100% al azar para descubrir cómo funciona la gravedad del juego. Entrenád primero (opción 1) y luego volved a visualizar.

¿Dónde guarda mi programa los resultados tras entrenar?

- grafica_aprendizaje.png (en la raíz del proyecto): Una imagen que dibuja la curva de aprendizaje de la IA.
- models/cerebro_cartpole.pth: El archivo que guarda los "pesos" y la memoria matemática de la red neuronal.

8.5 Solución de Problemas Comunes

Tabla 14. Solución de problemas comunes

Error / Problema	Solución
ModuleNotFoundError: No module named 'gymnasium'	El entorno Python no detecta las librerías. Ejecutad el comando de instalación del apartado 8.2.
La ventana del juego no se abre / error 'Headless'	Estáis ejecutando el script desde WSL o un servidor sin interfaz gráfica. Ejecutad desde CMD o PowerShell en Windows nativo.
El entrenamiento va demasiado lento	El entrenamiento consume mucho procesador. Cerrad otros programas pesados mientras la IA hace sus cálculos.
El péndulo cae siempre aunque haya entrenado	Puede que el modelo sea de una sesión temprana. Borrad models/cerebro_cartpole.pth y volved a entrenar desde cero.

9. Conclusiones

Puedo afirmar que el proyecto ha cumplido satisfactoriamente todos los objetivos iniciales marcados en mi Acta de Constitución. He logrado construir desde cero un ecosistema de Inteligencia Artificial capaz de aprender a dominar un entorno físico interactivo sin intervención humana.

A nivel personal y técnico, este proyecto ha supuesto un reto enorme. Pasar de la programación tradicional (donde yo le digo a la máquina exactamente qué hacer con sentencias if/else) a la programación neuronal (donde la máquina aprende sola a base de recompensas) me ha obligado a cambiar por completo mi forma de pensar.

Además, durante la fase de validación, me encontré con problemas reales del mundo del Machine Learning que tuve que solucionar sobre la marcha:

- Mínimos Locales: Observé cómo a veces el Agente se estancaba en estrategias mediocres. Tuve que jugar con la "Aleatoriedad" (épsilon) para obligarle a probar cosas nuevas y que así descubriera la estrategia perfecta.
- Falsos Positivos y Suerte: Al principio, el Agente parecía perfecto tras un par de partidas buenas, pero al probarlo en el entorno de "Visualización" fallaba estrepitosamente. Aprendí que una IA necesita demostrar consistencia, por lo que le exigí rachas de 10 partidas perfectas seguidas antes de dar el entrenamiento por bueno.
- Sobreajuste (Overfitting): Descubrí que entrenar a la máquina "de más" es malo. Si el Agente jugaba demasiadas veces de forma perfecta, su memoria olvidaba cómo recuperarse de los errores (Olvido Catastrófico) y su rendimiento caía en picado. Lo solucioné implementando un sistema de "Parada Temprana" (Early Stopping).

A nivel empresarial, el éxito de este prototipo (PoC) demuestra la viabilidad de mi propuesta inicial: automatizar las pruebas de QA en el desarrollo de videojuegos mediante Aprendizaje por Refuerzo es posible, reduce tiempos operativos y elimina la frustración del testeo manual continuo.

10. Posibles Mejoras

Aunque el prototipo es 100% funcional y resuelve el problema del entorno CartPole, en un entorno de producción real o para futuras versiones del proyecto, se podrían implementar las siguientes mejoras:

10.1 Cambio de Algoritmo (PPO)

Actualmente el sistema usa DQN (Deep Q-Network), que es excelente para decisiones discretas (pulsar izquierda o derecha). Para probar juegos más complejos donde haya que mover un joystick de forma suave (espacio de acciones continuo), habría que migrar el cerebro a algoritmos matemáticos más modernos como PPO (Proximal Policy Optimization).

10.2 Visión Artificial por Pantalla

Ahora mismo, la IA "hace trampa" porque lee las variables directamente de la memoria RAM del juego. Una mejora brutal sería integrar librerías como OpenCV para que la IA tome decisiones mirando únicamente los píxeles de la pantalla, simulando exactamente a un jugador humano.

10.3 Interfaz Gráfica (GUI)

Sustituir el menú por consola actual por una interfaz gráfica moderna (desarrollada con Tkinter, PyQt o como una web app) donde los diseñadores de niveles puedan subir sus juegos, ajustar los parámetros con barras deslizantes y ver las gráficas generarse en tiempo real.

10.4 Mejoras Técnicas del Algoritmo

- Target Network: Añadir una segunda red neuronal que se actualice cada N pasos para eliminar la inestabilidad del entrenamiento. Es la mejora técnica más importante pendiente.
- Fichero de configuración (config.yaml) para centralizar todos los hiperparámetros sin modificar el código fuente.
- Integración continua con GitHub Actions para ejecutar los tests automáticamente en cada push al repositorio.

11. Bibliografía y Enlaces

Para el desarrollo de este proyecto, tanto en su fase de investigación teórica como en la implementación técnica, me he apoyado en la siguiente documentación y herramientas oficiales:

11.1 Documentación Oficial

Fundación Farama (Gymnasium) — Documentación oficial del entorno CartPole y las mecánicas de simulación física:

https://gymnasium.farama.org/

PyTorch Foundation — Referencia oficial sobre tensores, funciones de pérdida (MSELoss) y optimizadores (Adam):

https://pytorch.org/docs/stable/index.html

Matplotlib — Guía de uso para la representación gráfica de datos de rendimiento:

https://matplotlib.org/stable/contents.html

11.2 Estudios de Sector

Modl.ai — State of Games QA Report: Estudio sobre los cuellos de botella en el control de calidad de videojuegos que inspiró la visión de negocio de este proyecto:

https://modl.ai/state-of-games-qa-report/

11.3 Repositorio del Proyecto

Código fuente completo del proyecto en GitHub:

https://github.com/Maaxii00/Proyecto-Intermodular